AULA 05: ESTRUTURAS DE REPETIÇÃO

Profª. Drª. Tatiele Bolson Moro

Estruturas de repetição

- As estruturas de repetições são bastante importantes pois muitas vezes necessitamos executar um mesmo procedimento diversas vezes.

- A linguagem C basicamente possui três tipos de estrutura de repetição:
  - FOR, WHILE e DO WHILE

For

- O comando `for` permite implementar **laços de repetição**, ou seja, definir trechos do código de um programa que se repetem.

- O `for`, como qualquer iteração, necessita de uma **variável** para controlar a quantidade de loops (voltas) a serem executadas.

- O comando `for` pode incluir na sua própria definição uma **variável contadora**:
  - A variável que vai definir quantas vezes aquele trecho de código será executado.

For

- No comando `for`, a **variável contadora** deverá ser **inicializada** com um valor, e ser **incrementada** ou **decrementada** a cada loop;

- Além disso, deveremos ter uma **condição**, **testando se** a quantidade de vezes que o **laço** deveria ser executado **chegou ao fim ou não** (dependendo do valor atual da variável contadora);

For

- Exemplo:
  - Executando um laço de repetição **4 vezes**.
    - No `for`, inicializamos uma variável contadora `int i`, cujo valor inicial é `0`;
    - A cada loop/laço a variável é **incrementada em 1**;
    - Após cada incremento, é testada uma condição para verificar se a variável chegou ao fim, ou seja, se o seu valor é igual a 4:
      - Caso o valor tenha chegado a 4, o laço termina;
      - Caso contrário, o laço se repete novamente.

For

- Portanto, o comando for necessita de 3 partes:

```
for(valor inicial; condição; incremento ou decremento)
{
     <bloco de instruções>;
}
```

- Em que:
  - **Valor inicial:** definimos um valor inicial para nossa variável contadora, que no caso do exemplo anterior, definimos que i vale 1 (i=0);
  - **Condição:** testamos se o laço chegou ao fim, que no caso do exemplo anterior, se i chegou a 4 (i < 4);
  - **Incremento:** i é incrementado em 1, para contar o número de vezes que o laço será repetido (i++)
  - **Bloco de instruções:** bloco que será executado 4 vezes, no exemplo.

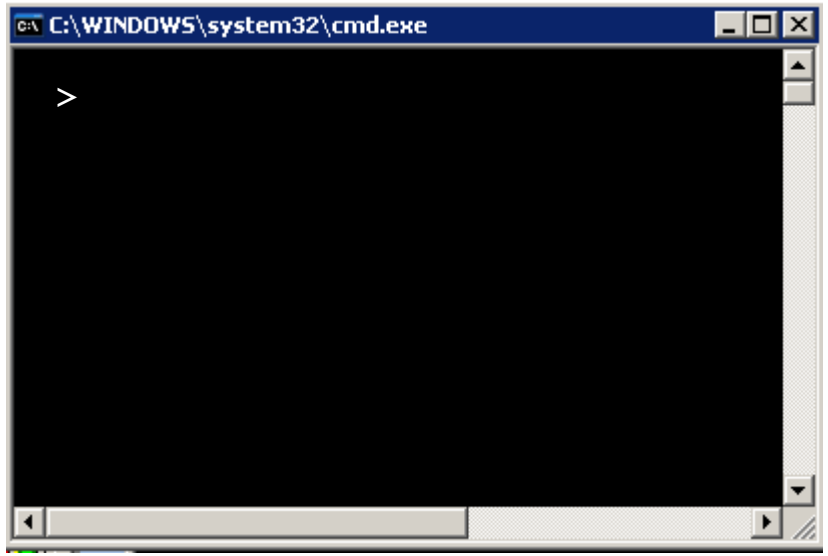
For

- Desta forma, o exemplo anterior ficaria assim:

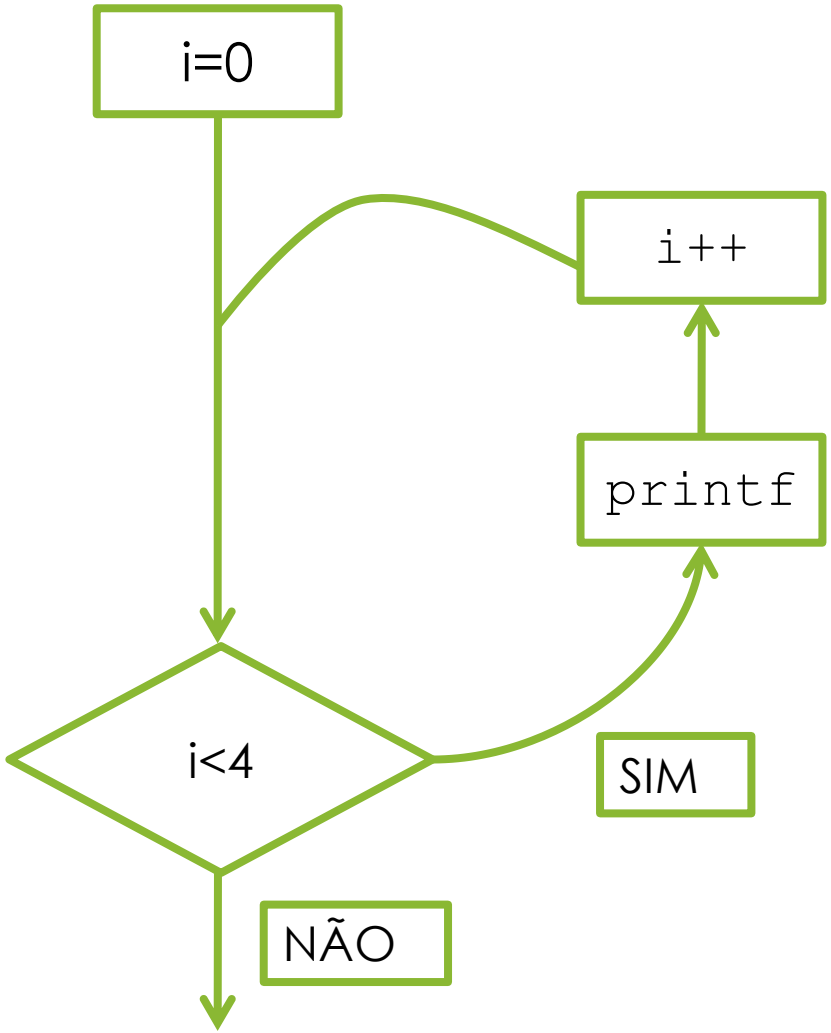
```
main()
{
    int i;
    for(i=0; i<4; i++)
    {
        //bloco de instruções
    }
}
```

Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

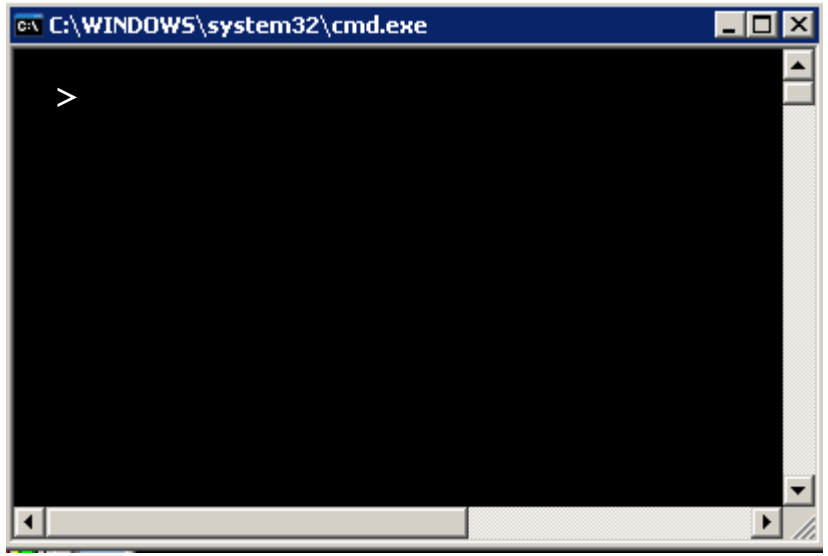


- Ordem de execução:

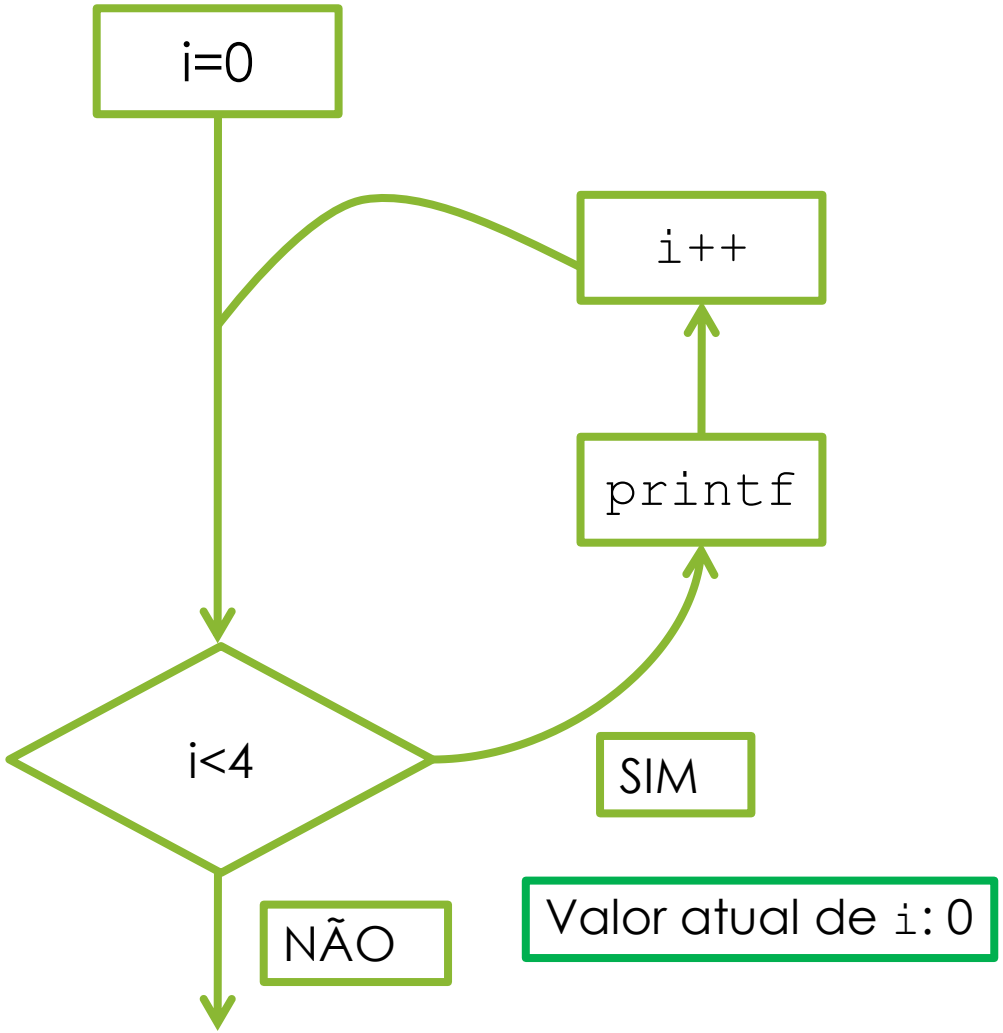


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

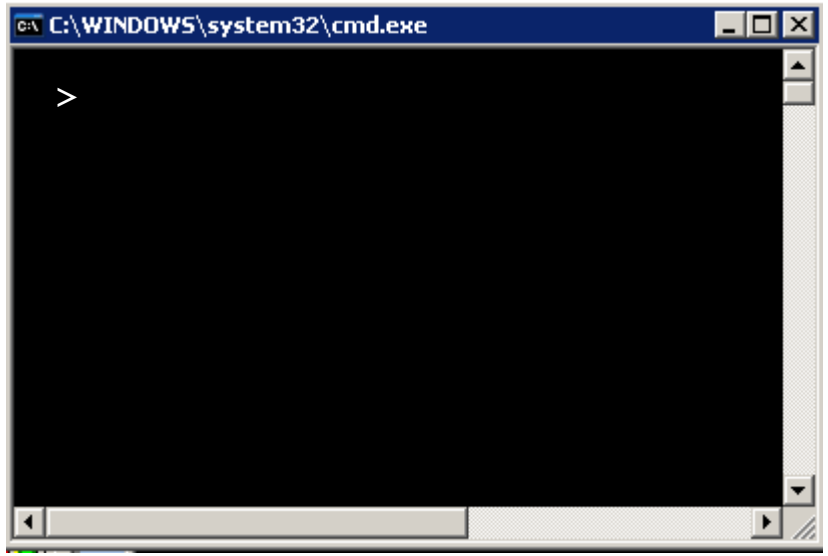


- Ordem de execução:

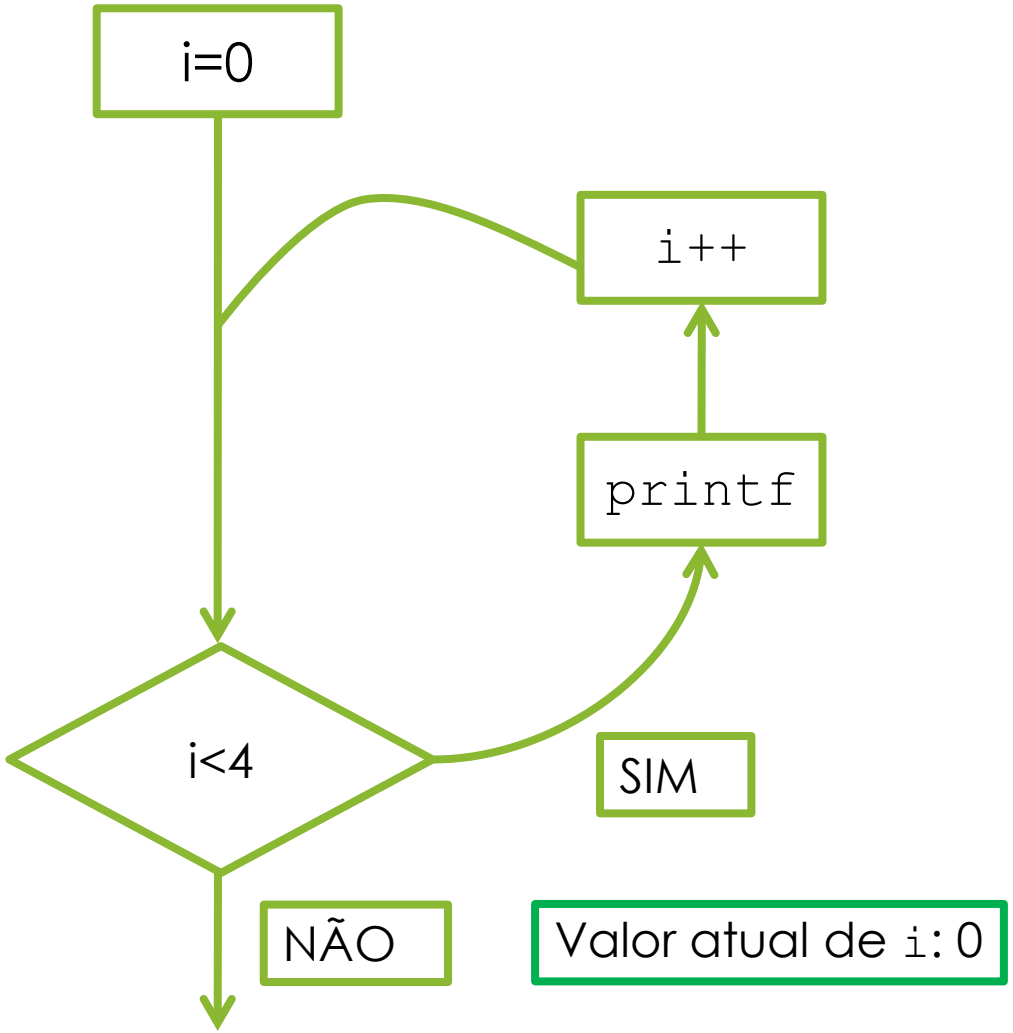


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

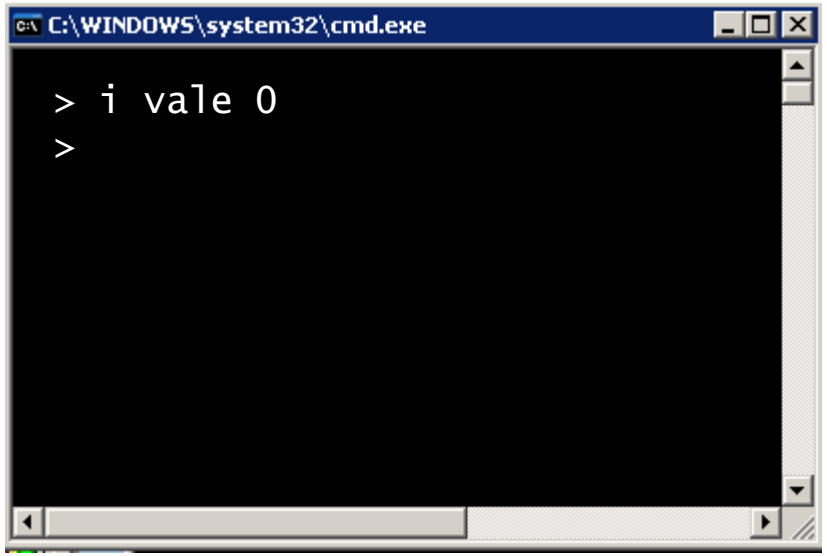


- Ordem de execução:

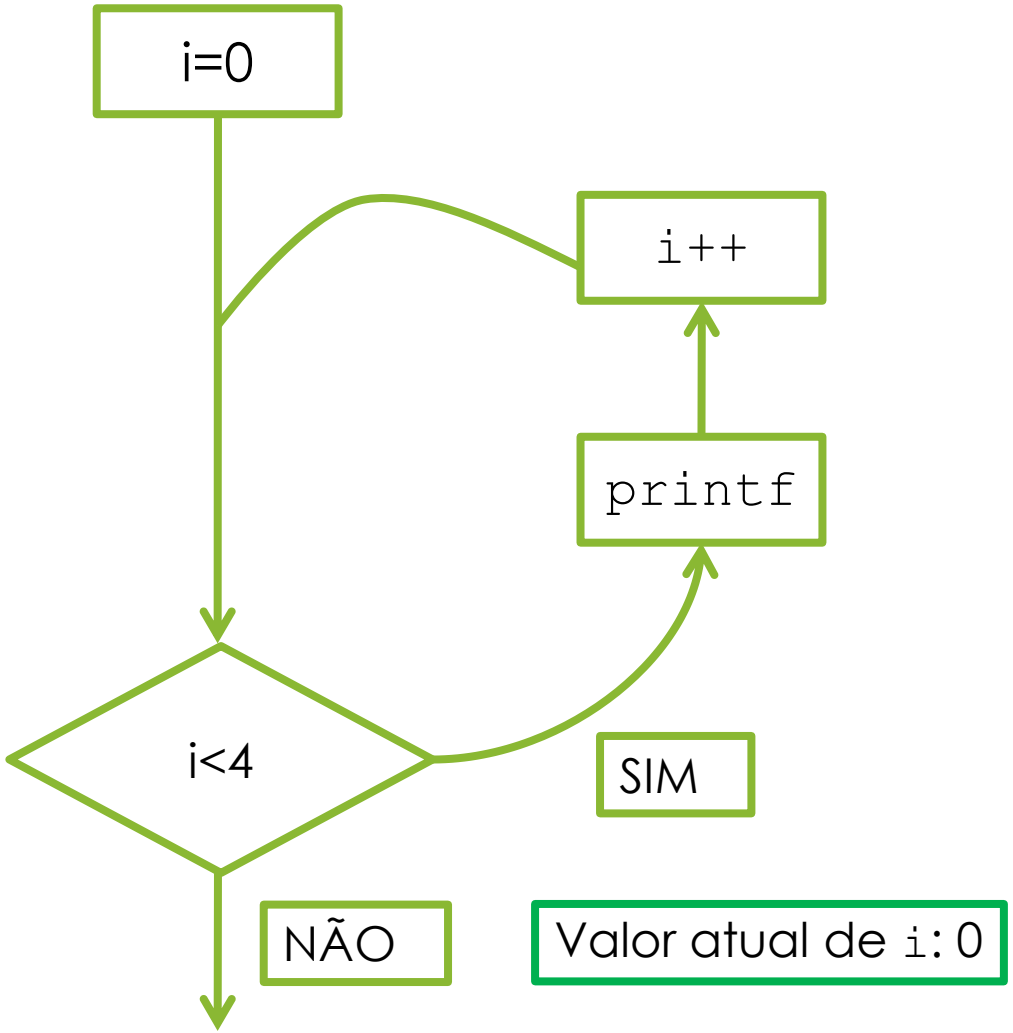


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
          printf("i vale %d\n",i);
     }
}
```

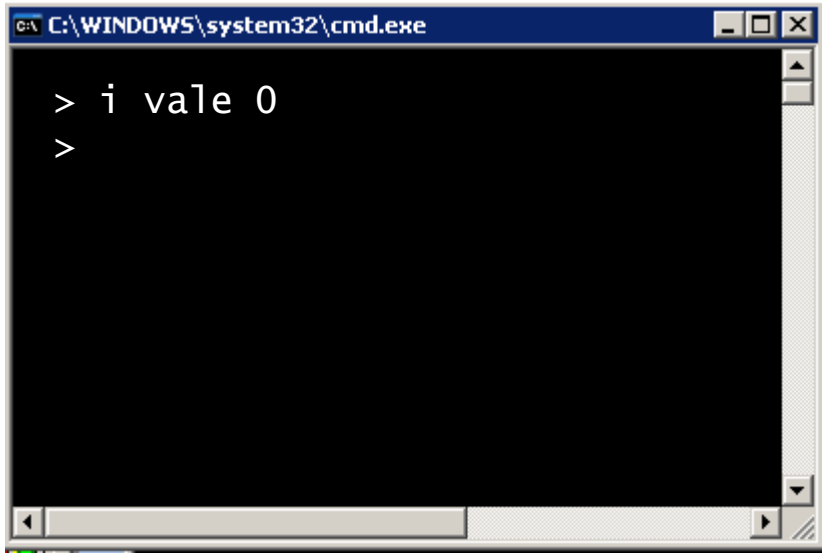


- Ordem de execução:

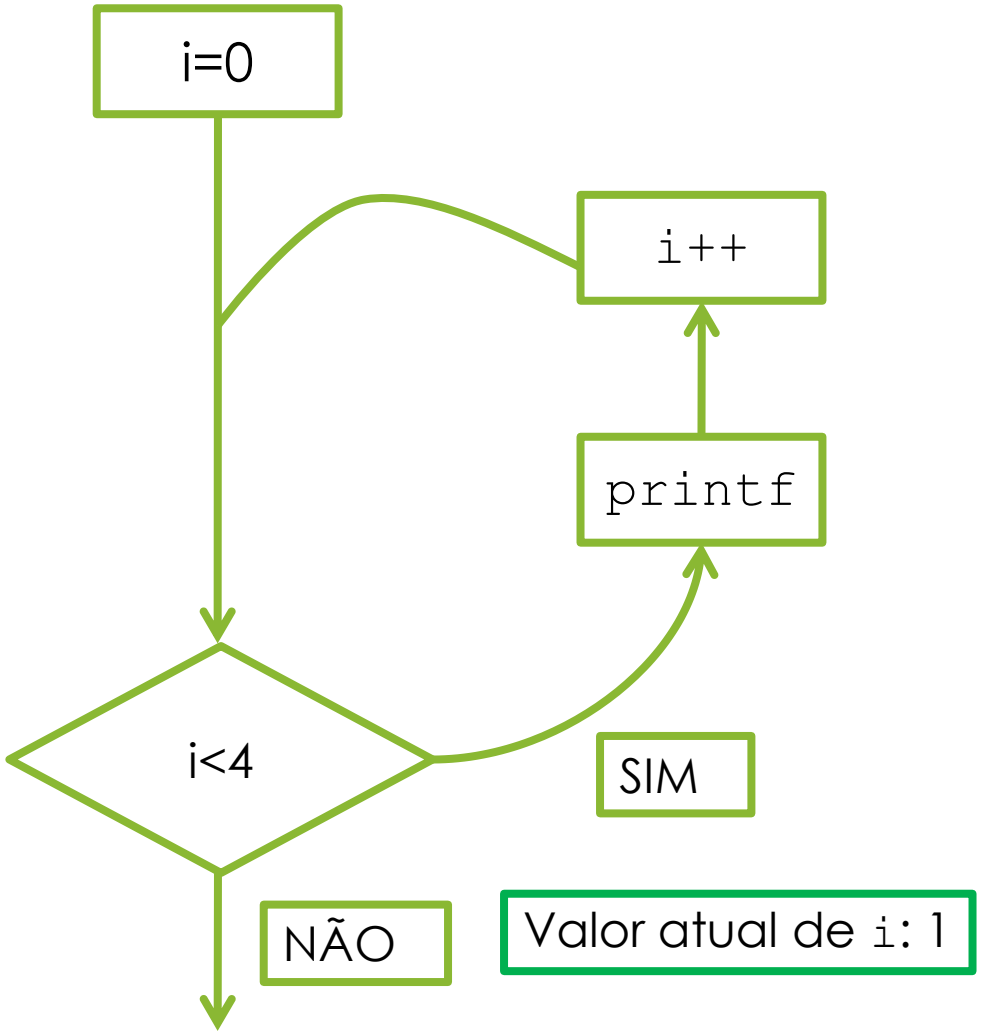


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

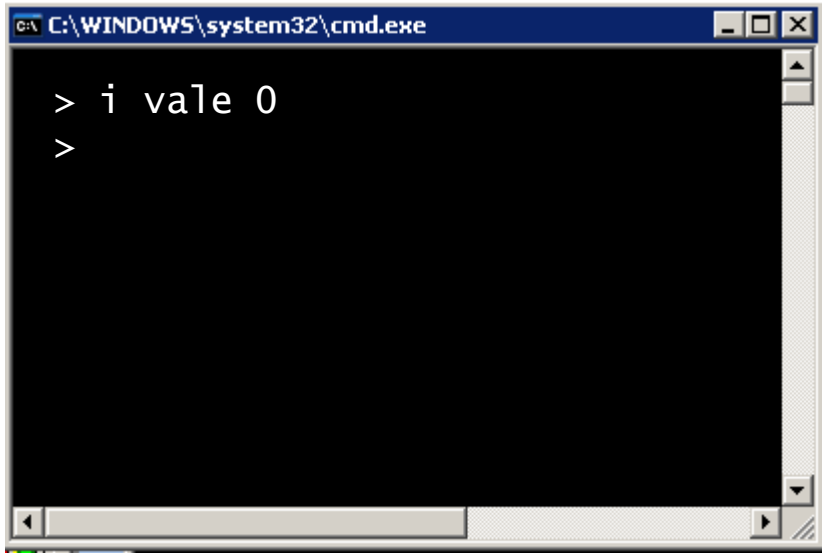


- Ordem de execução:

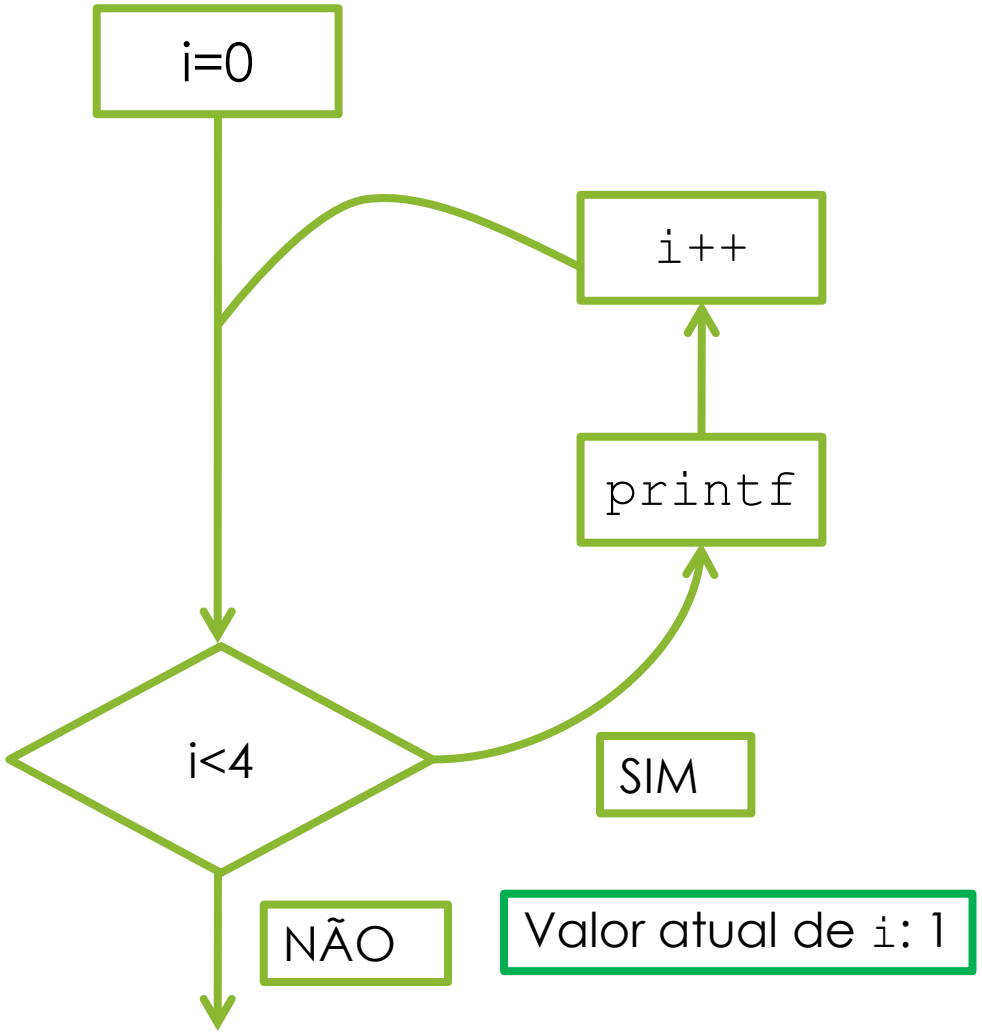


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0; i<4; i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

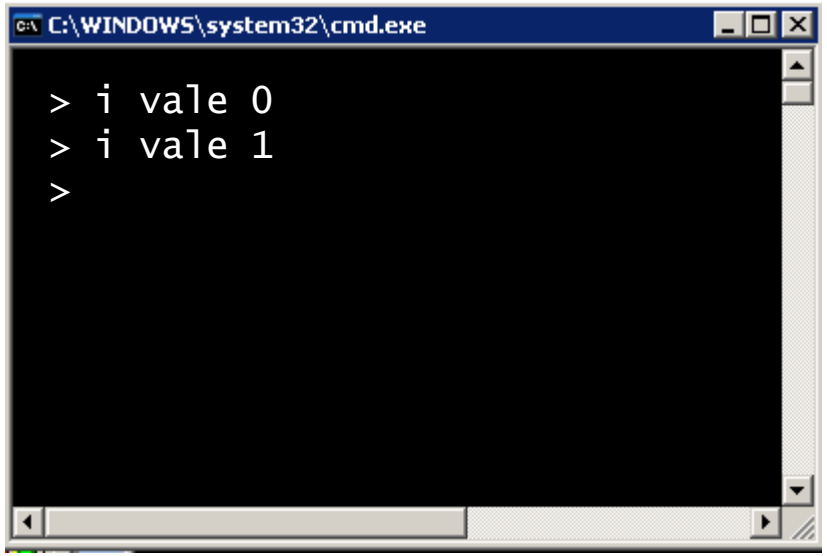


- Ordem de execução:

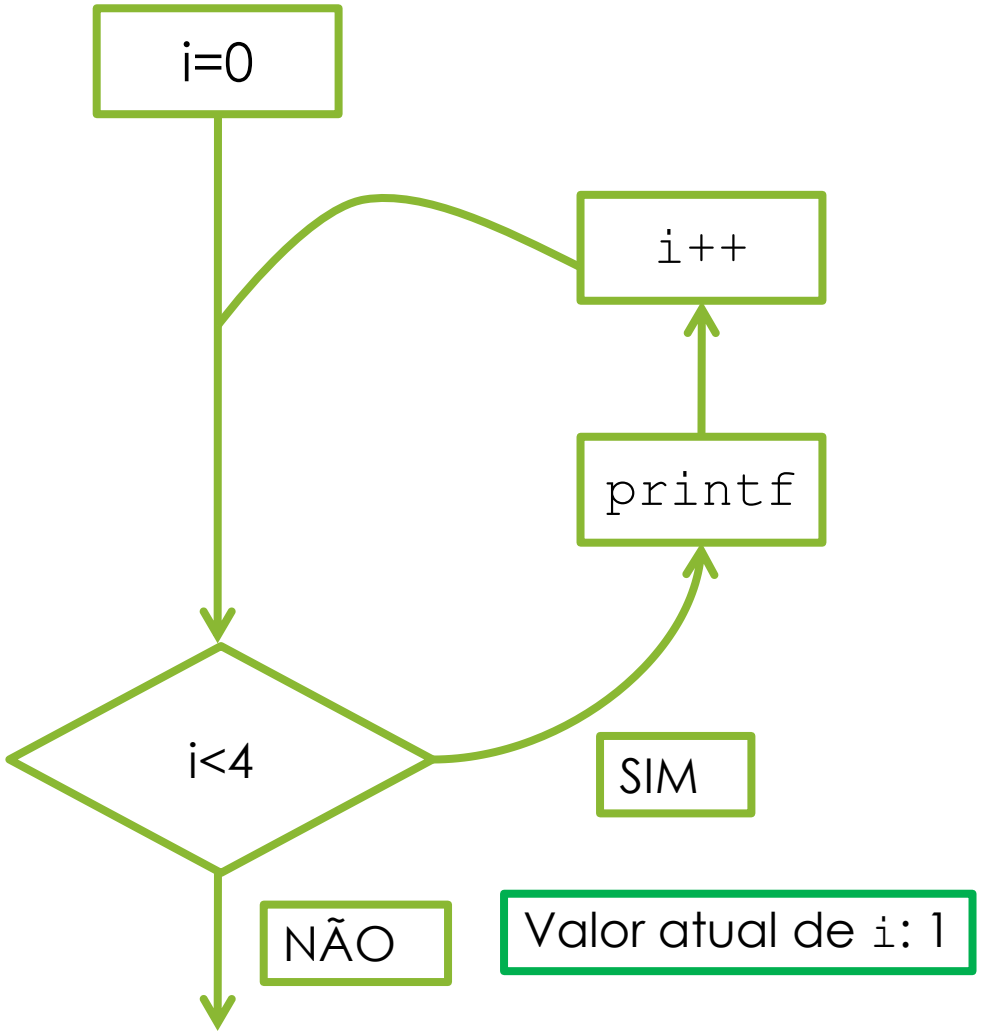


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
          printf("i vale %d\n",i);
     }
}
```

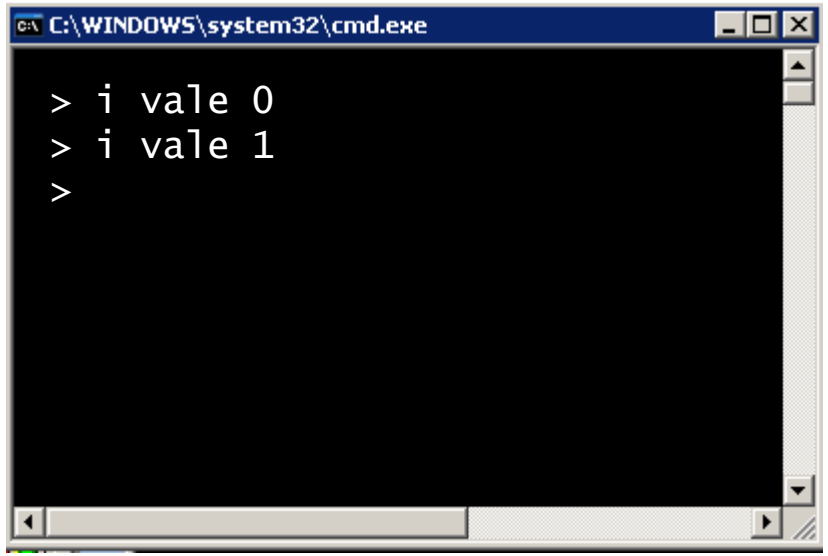


- Ordem de execução:

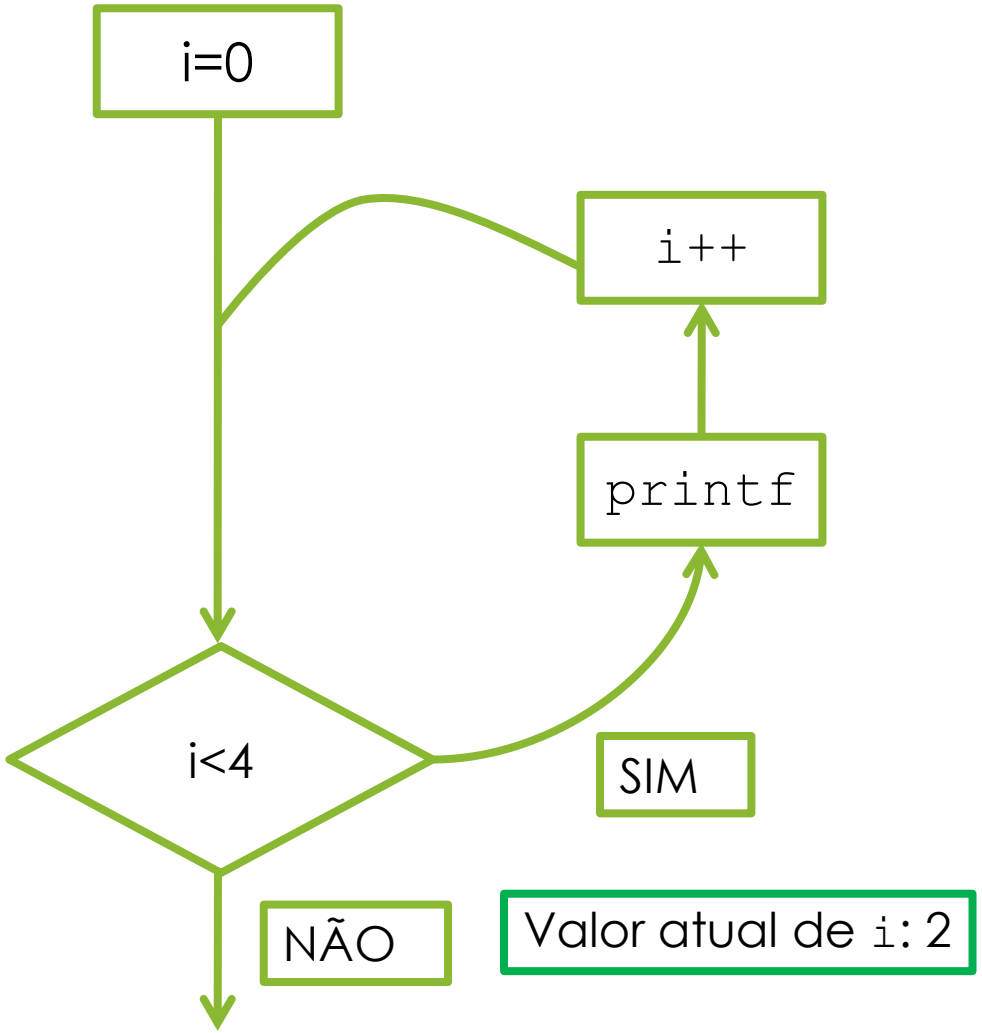


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

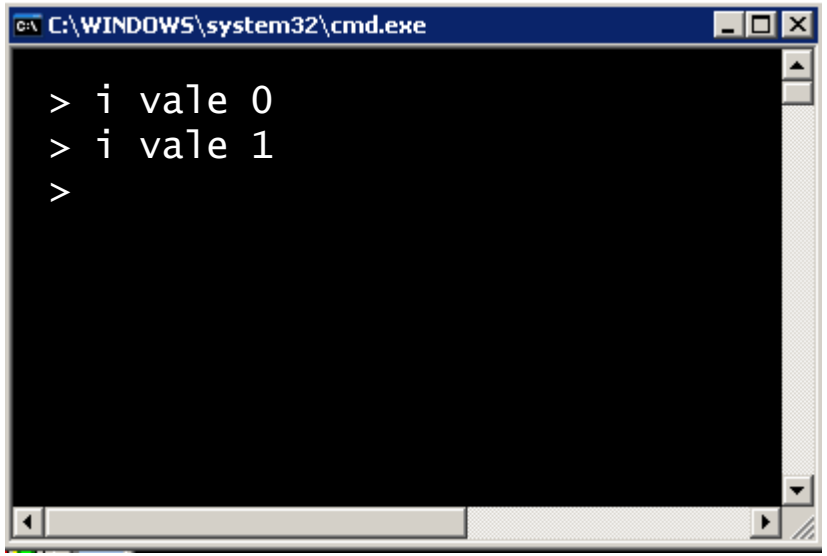


- Ordem de execução:

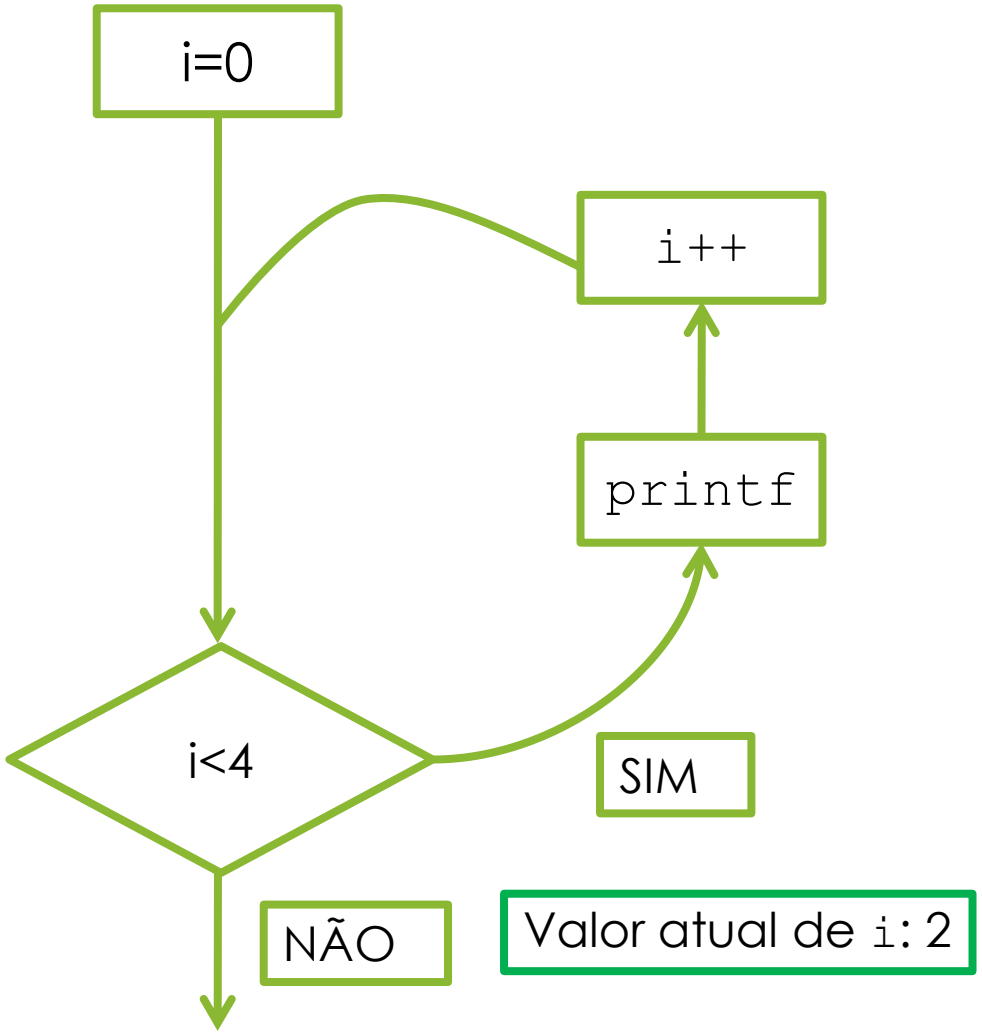


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0; i<4; i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

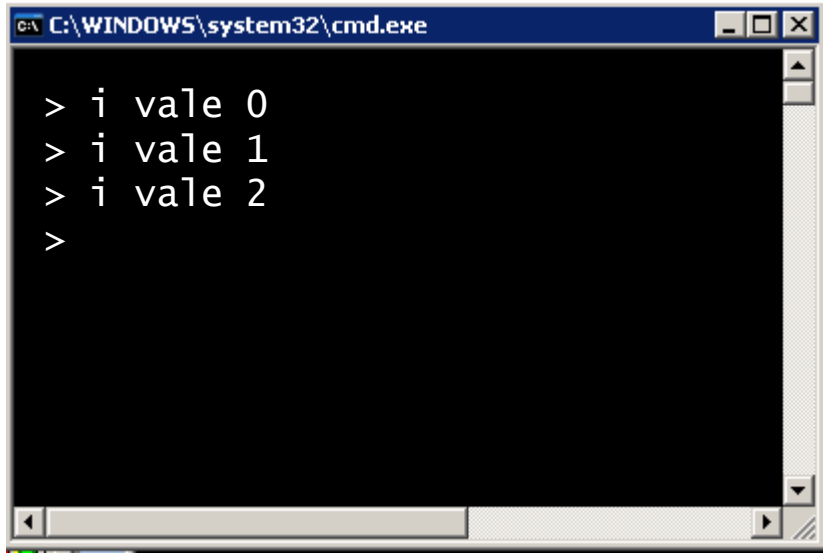


- Ordem de execução:

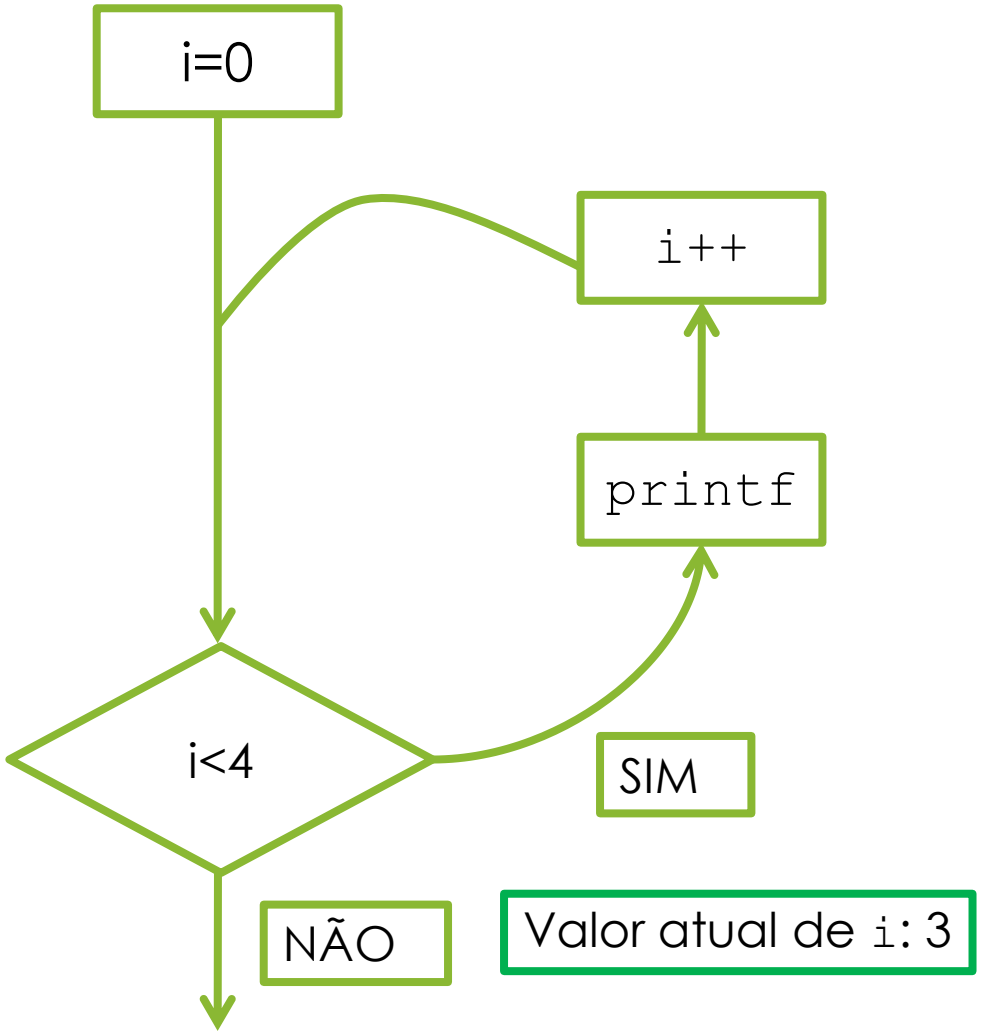


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
          printf("i vale %d\n",i);
     }
}
```

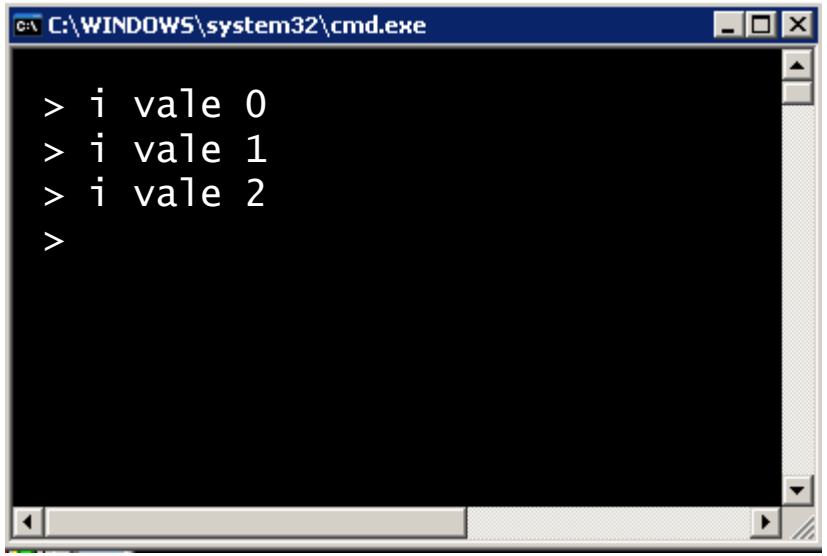


- Ordem de execução:

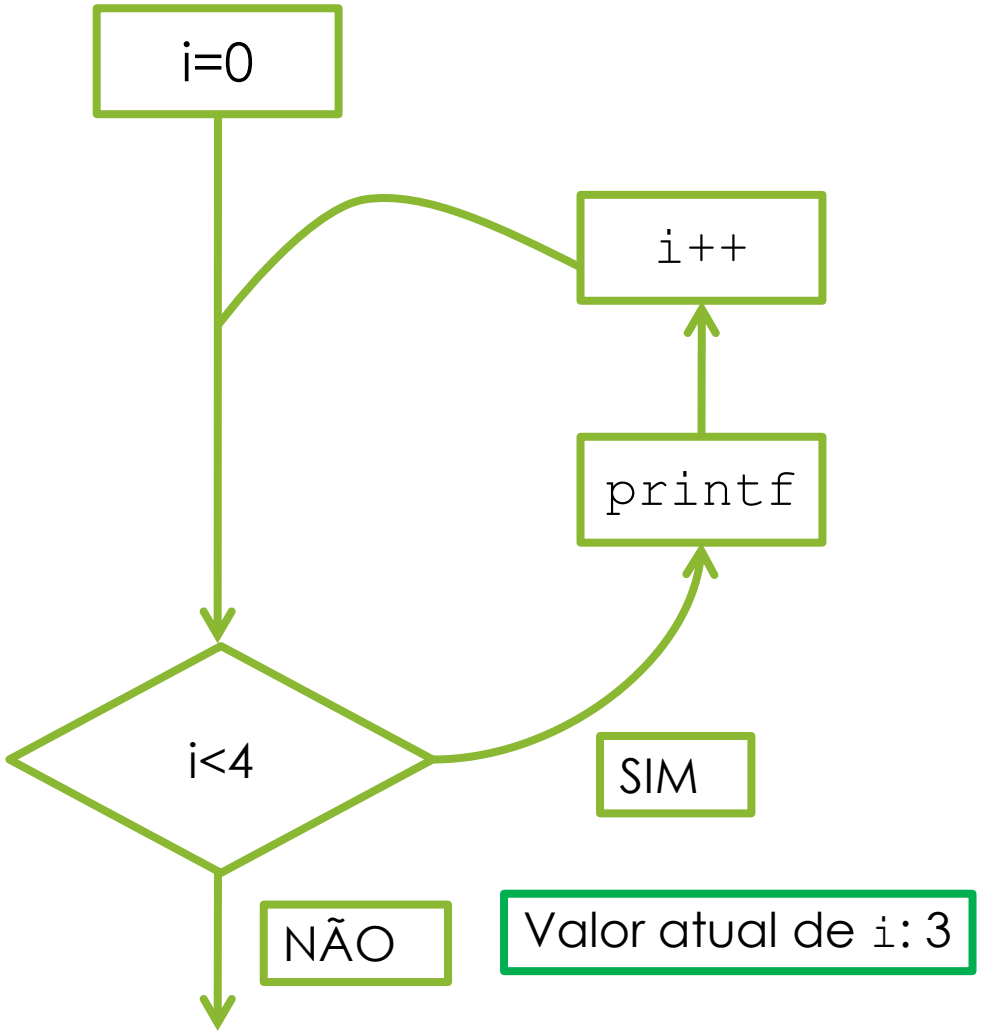


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

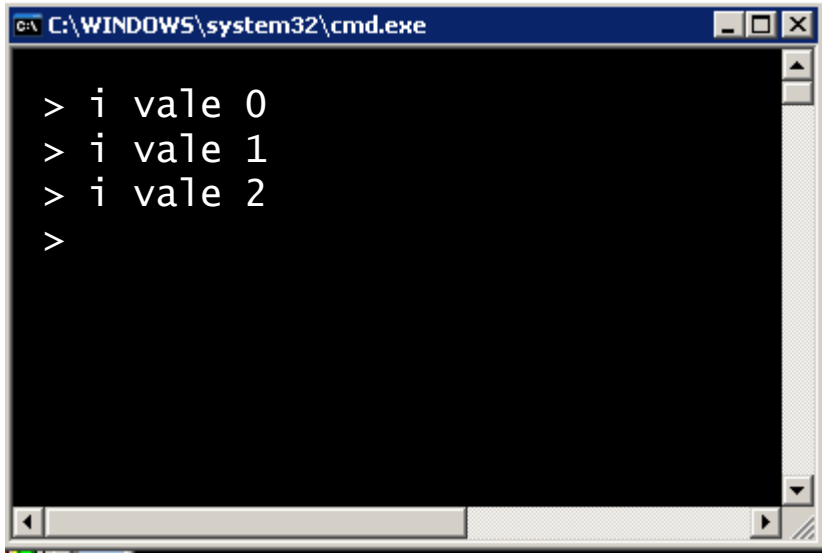


- Ordem de execução:

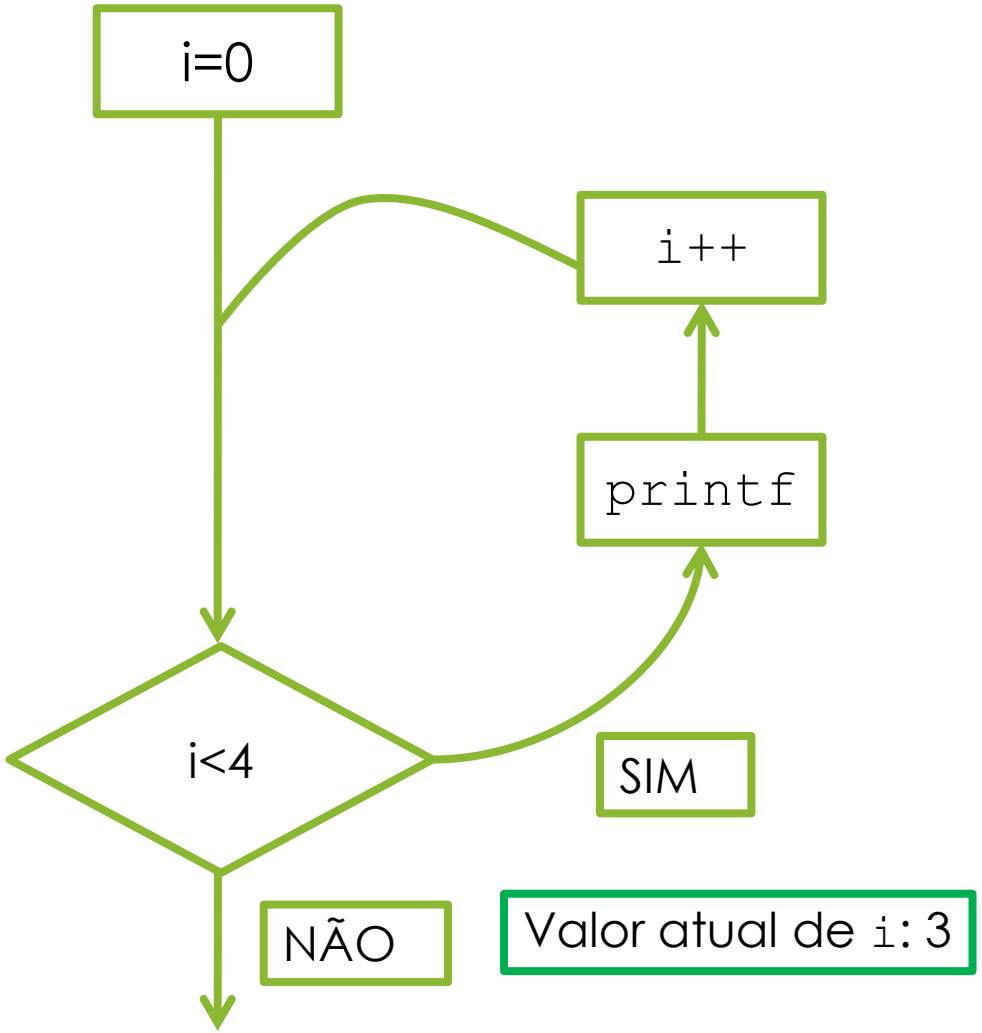


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

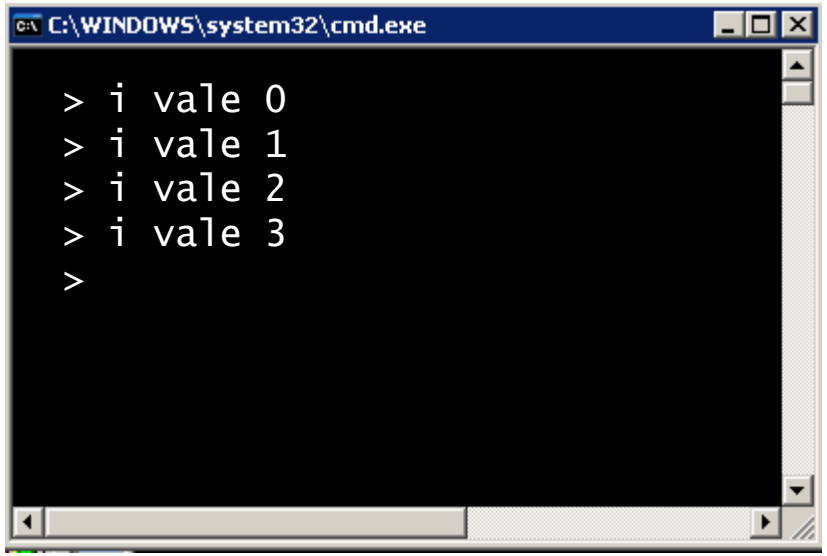


- Ordem de execução:

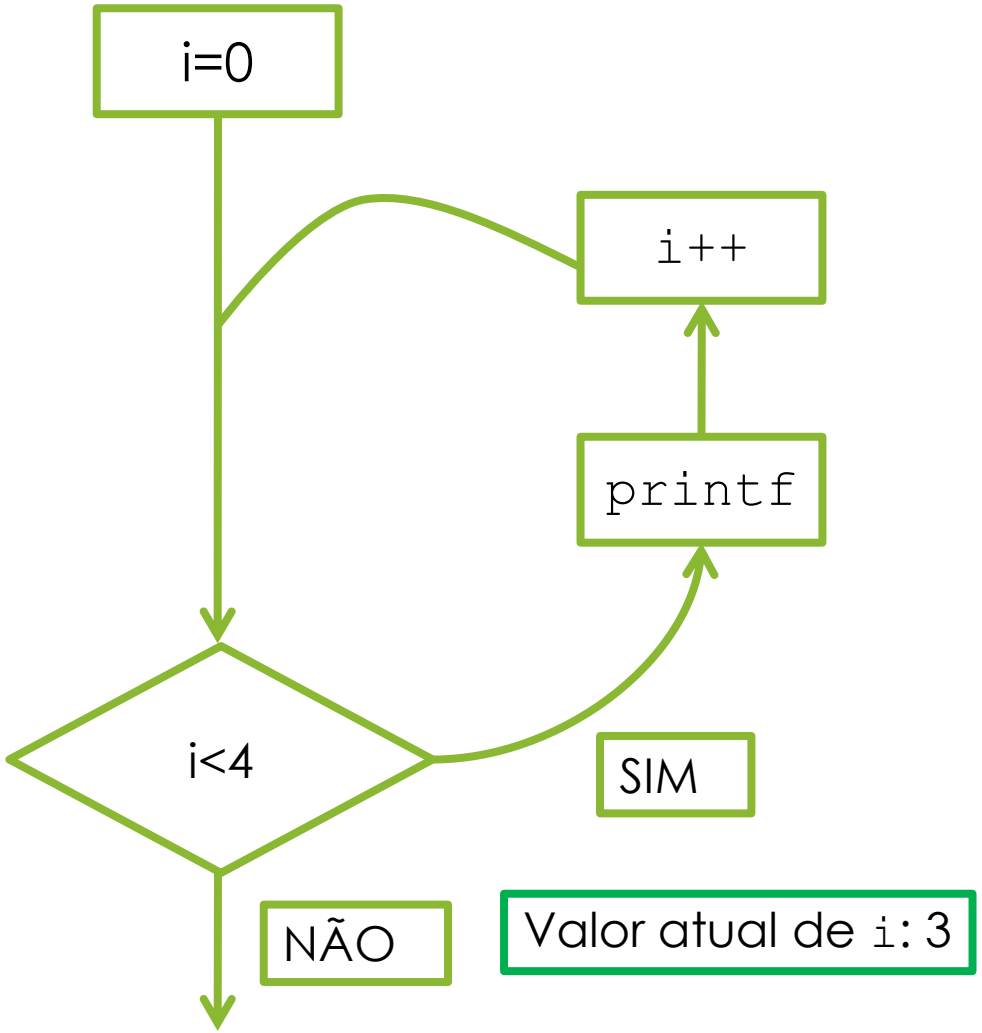


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
          printf("i vale %d\n",i);
     }
}
```

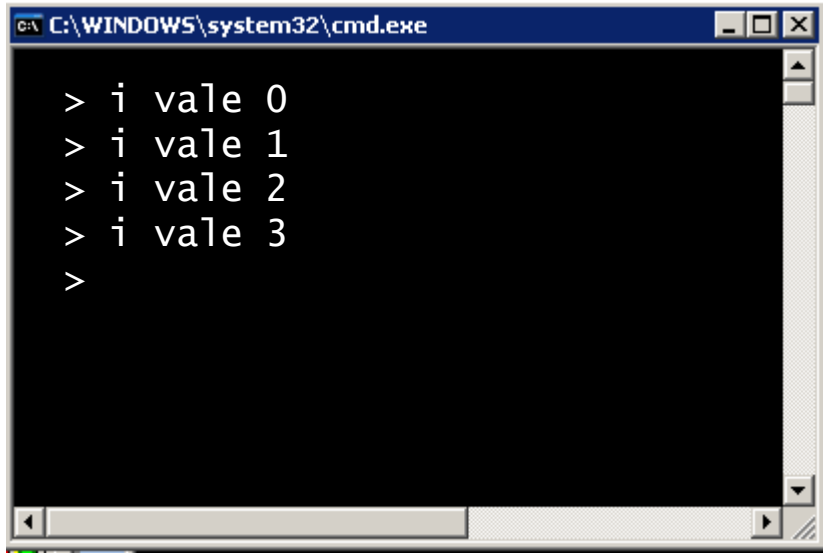


- Ordem de execução:

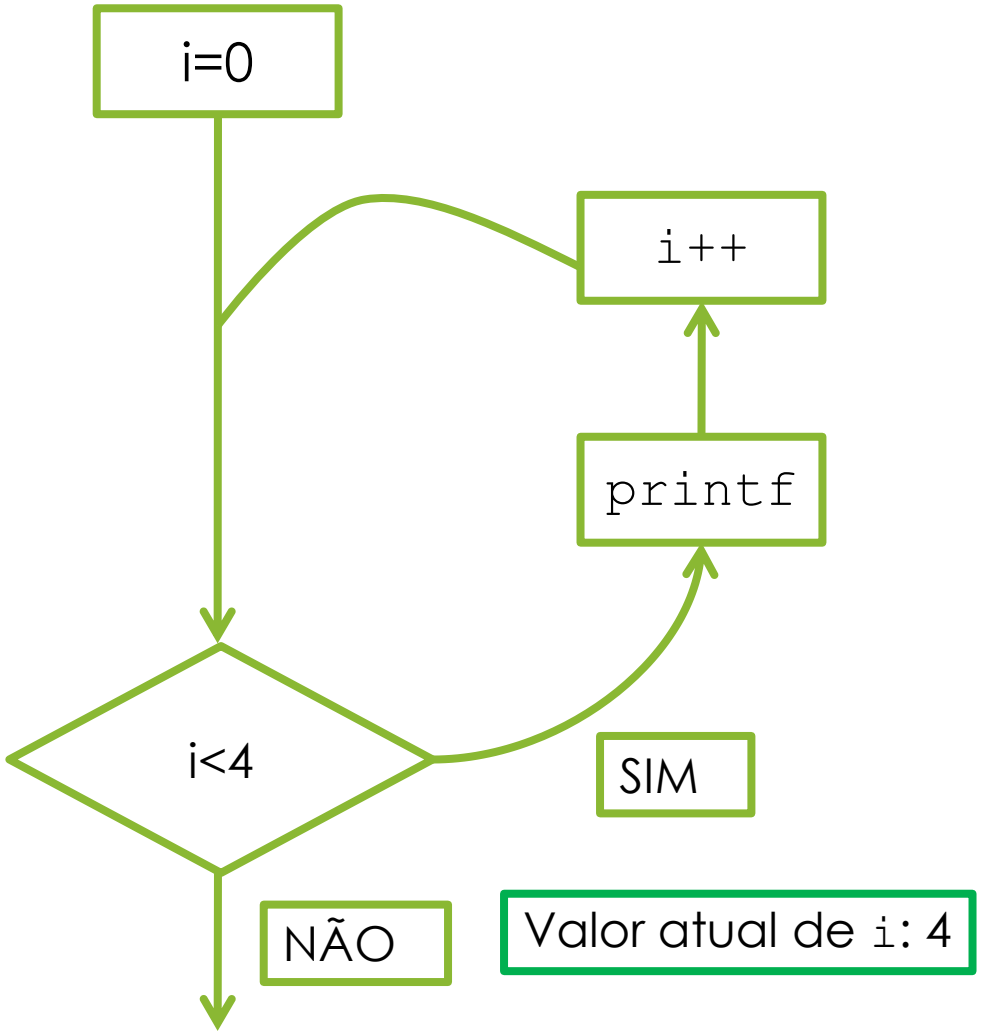


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

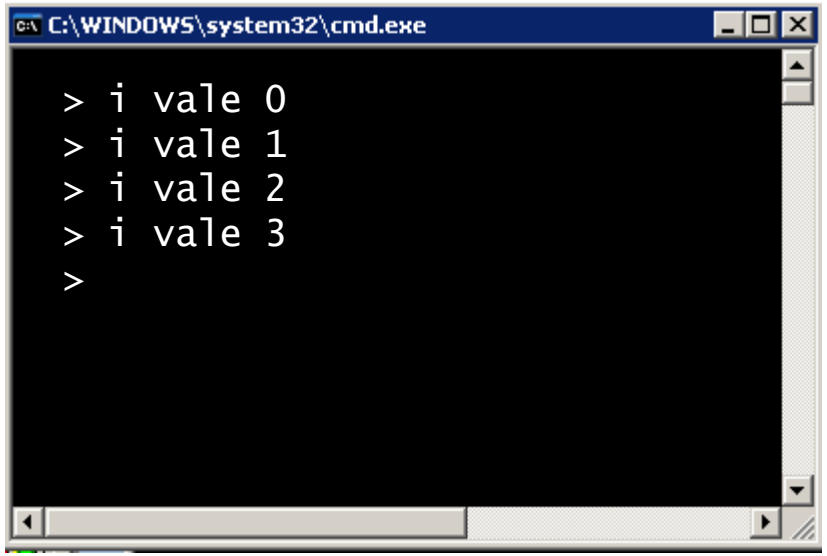


- Ordem de execução:

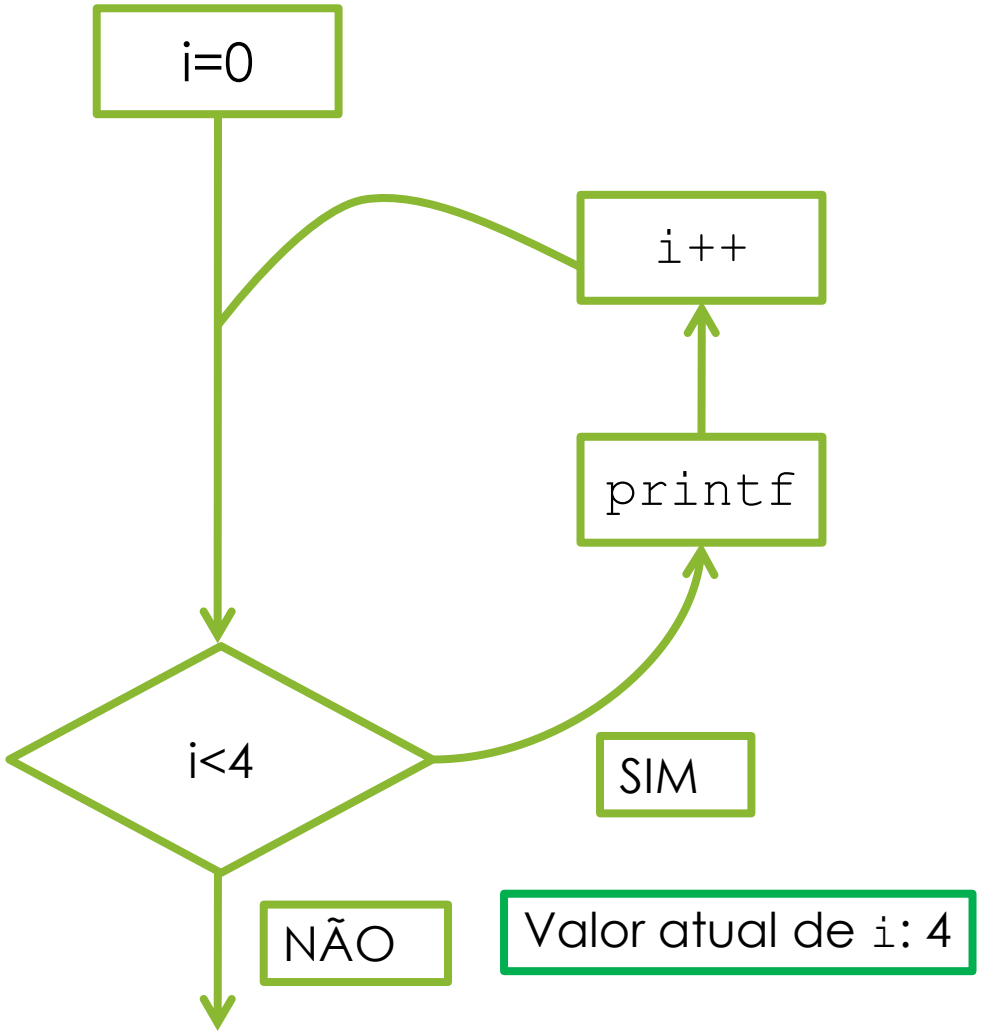


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```

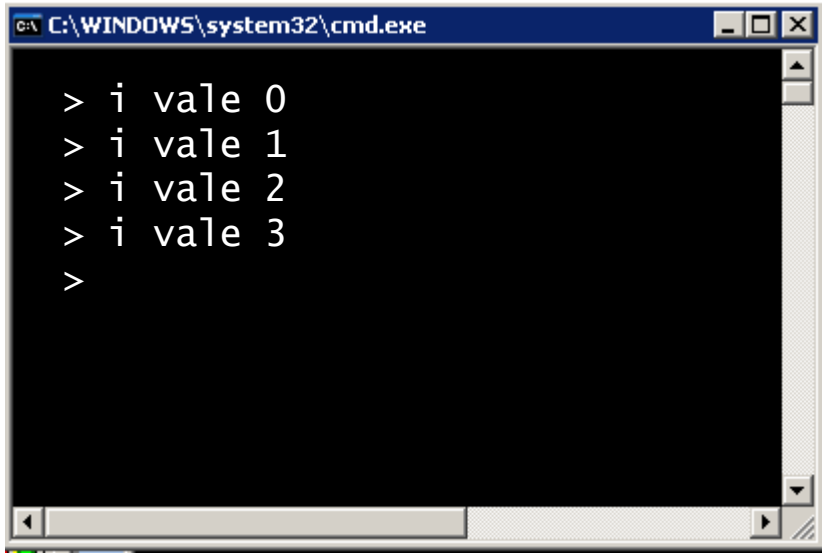


- Ordem de execução:

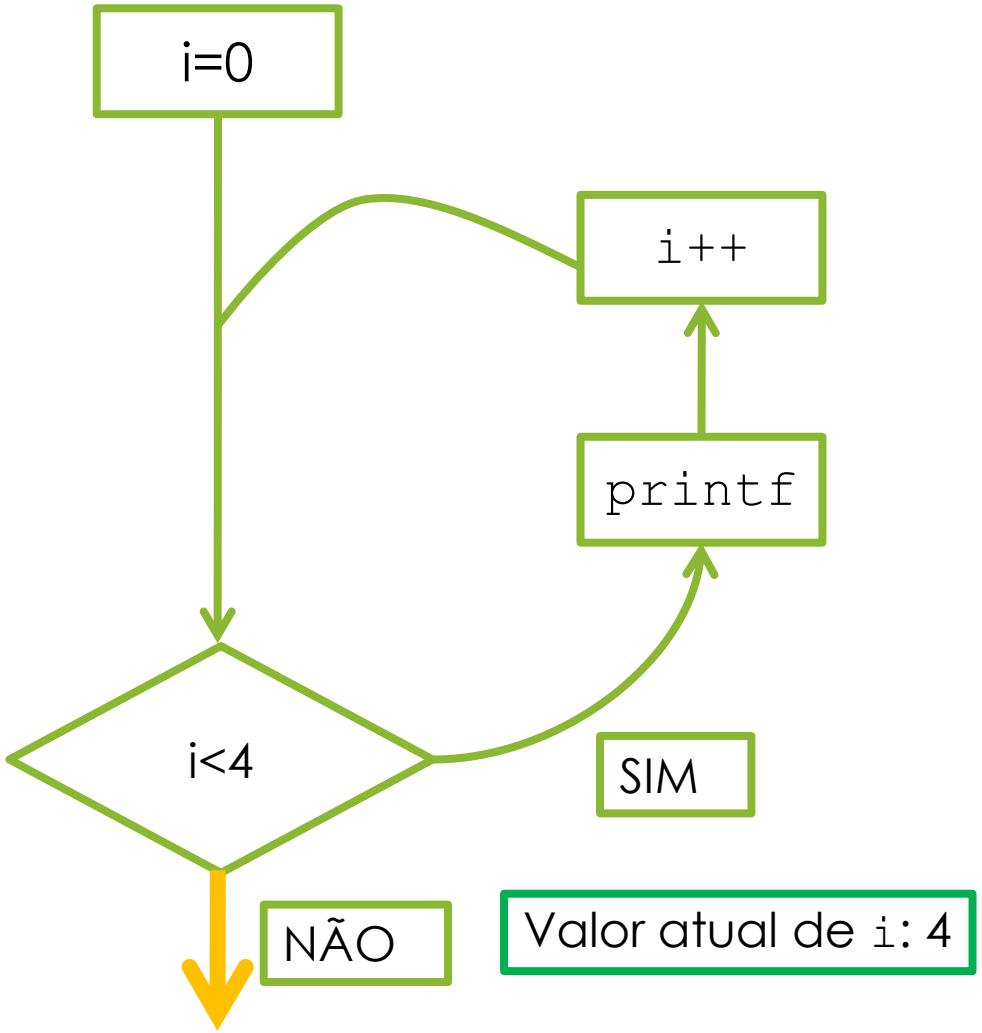


Entendendo o Exemplo 1

```
main()
{
    int i;
    for( i=0;  i<4;  i++ )
     {
            printf("i vale %d\n",i);
     }
}
```



- Ordem de execução:



Exemplo 2 - incrementando

```
#include <stdio.h>
main()
{
    int i, max=10;
    for(i=0; i<max; i++)
    {
        printf("i eh %d\n",i);
    }
}
```

Exemplo 3 - decrementando

```
#include <stdio.h>
main()
{
    int i, min=0;
    for(i=10; i>min; i--)
    {
        printf("i eh %d\n",i);
    }
}
```

Exemplo 4 – feliz ano novo

```
#include <stdio.h>
main()
{
 int waits = 10;
 int i;
 for(i = waits; i > 0;i--)
 {
     printf("%d\n",i);
 }
 printf("Feliz ano novo!!\n");
}
```

Exemplo 5 – incrementando de 3 em 3

```
#include <stdio.h>
main()
{
    int i;
    for(i=0; i<10; i+=3)
    {
        printf("i eh %d\n",i);
    }
}
```

Exemplo 6 – decrementando de 3 em 3

```
#include <stdio.h>
main()
{
    int i;
    for(i=10; i>0; i-=3)
    {
        printf("i eh %d\n",i);
    }
}
```

Exemplo 7 – duas variáveis

```
#include <stdio.h>
main()
{
    int i,j;
    for(i=0, j=100; i!=j ; i++, j--)
    {
        printf("i eh %d e j eh %d\n",i,j);
    }
}
```

Siiimmm, podemos inicializar quantas variáveis desejarmos, testar quantas condições quisermos, incrementar, decrementar, fazer...

TUDO SEPARADO POR VÍRGULA!!!

Exemplo 8 – sem inicialização

```
#include <stdio.h>
main()
{
    int i;
    i=0;
    for(; i<10; i++)
    {
        printf("i eh %d\n",i);
    }
}
```

Exemplo 9 – sem incremento

```
#include <stdio.h>
main()
{
    int i;
    for(i=0; i<10; )
    {
        printf("i eh %d\n",i);
        i++;
    }
}
```

Exemplo 10 – sem inicialização nem incremento

```
#include <stdio.h>
main()
{
	int i;
	i=0;
	for(; i<10; )
	{
		printf("i eh %d\n",i);
		i++;
	}
}
```

Exemplo 11 – cuidado com loops infinitos

```
#include <stdio.h>
main()
{
  int i;
  for(i=0; i<10; i-- )
  {
     printf("i eh %d\n",i);
  }
}
```

Exemplo 13 – cuidado com loops infinitos v2

```
#include <stdio.h>
main()
{
    int i;
    for(i=0; 1; i++ )
    {
        printf("i eh %d\n",i);
    }
}
```

Lembram? **0** é o mesmo que **falso**, e qualquer **número diferente de zero** é o mesmo que **verdadeiro**!

Exemplo 14 – interagindo com o usuário

```
#include <stdio.h>
main()
{
    int i,n;
    printf("Quantas vezes voce quer repetir o laco?");
    scanf("%d",&n);
    for(i=0; i<n; i++ )
    {
        printf("repeticao %d\n",i);
    }
}
```

Exemplo 15 – interagindo dentro do laço

```
#include <stdio.h>
main()
{
    int i,n,variavel;
    printf("Quantas vezes voce quer repetir o laco?");
    scanf("%d",&n);
    for(i=0; i<n; i++ )
    {
        printf("Digite um valor qualquer: ");
        scanf("%d",&variavel);
        printf("obrigado por digitar %d na interacao %d\n",variavel,i);
    }
}
```

Exemplo 16 – quebrando o laço

```
#include <stdio.h>
#include <stdbool.h>
main()
{
    int i, op;
    for(i=0; true; i++ )
    {
        printf("interacao %d\n",i);
        printf("Quer continuar? ");
        scanf("%d",&op);
        if(op==0)
        {
            printf("Obrigado pela presenca neste laco de repeticao");
            break;
        }
    }
}
```

Biblioteca **<stdbool.h>** - manipula variáveis lógicas, como verdadeiro e falso, ou seja true e false

O comando **break** faz sair do laço, indo para a próxima linha após o bloco **for.**

Exemplo 17 – quebrando/continuando no laço

```
#include <stdio.h>
#include <stdbool.h>
main()
{
  int i, op;
  for(i=0; true; i++ )
  {
        printf("interacao %d\n",i);
        printf("Quer continuar? ");
        scanf("%d",&op);
        if(op!=0)
        {
              continue;
              printf("sera que vou ser executado?\n");
        }
        else
        {
               printf("Ja vai tarde...\n");
               break;
        }
        printf("e eu sera que serei executado???\n");
  }
}
```

E comando **`continue`** manda seguir no laço para a próxima interação direto, ou seja, **vai direto pro incremento/decremento**, ignorando todo o resto do bloco!;

Exemplo 18 – media de notas da turma

```
#include <stdio.h>
main()
{
    int i, n;
    float media=0, nota;
    printf("Calculo da media de notas da turma em uma avaliacao \n\n");
    printf("Quantos alunos realizaram a avaliacao? ");
    scanf("%d", &n);
    for(i=1;i<=n;i++)
    {
        printf("Digite a nota do aluno %d: ", i);
        scanf("%f", &nota);
        media += nota;
    }
    media = media/n;
    printf("\nA media final da turma eh %.2f\n", media);
}
```

Exemplo 19 – repetindo o programa

```
#include <stdio.h>
#include <math.h>
main()
{
 int op, num;
 for(op=1;op!=0;)
 {
      printf("Digite algum numero: ");
      scanf("%d",&num);
      num = pow(num,2)*3;
      printf("O resultado da conta aleatoria eh: %d\n",num);
      printf("Deseja continuar?\n>>");
      scanf("%d",&op);
 }
}
```

Exemplo 20 – `for` dentro de um `for`

```
#include <stdio.h>
main()
{
    int i,j;
    for(i=0; i<20; i++)
    {
        printf("i:%d\n",i);
        for(j=0; j<10; j++)
        {
            printf("     j:%d\n",j);
        }
    }
}
```

Exemplo 21 – `for` dentro de um `for` dentro de um `for`

```
#include <stdio.h>
main()
{
    int i,j,k;
    for(i=0; i<20; i++)
    {
       printf("i:%d\n",i);
       for(j=0; j<10; j++)
       {
            printf("      j:%d\n",j);
          for(k=0; k<15; k++)
          {
               printf("         k:%d\n",j);
           }
       }
    }
}
```